

Module 5

Q. Spectral Clustering in Machine Learning

=>

1. Introduction

1. **Spectral Clustering** is an advanced **unsupervised learning algorithm** that groups data points based on **connectivity** rather than just distance.
2. Unlike algorithms like **K-Means**, which use **compactness (distance)** between data points, spectral clustering focuses on **how well the data points are connected**.
3. It can cluster two data points together even if they are far apart, as long as there is a **path of connected points** between them.
4. This makes it especially useful for identifying **non-linear or complex-shaped clusters**.
5. Example – Two circular or spiral-shaped clusters that are connected internally but distant in terms of Euclidean distance can be separated using spectral clustering.

2. Concept and Working

1. Spectral Clustering is based on the idea of a **graph representation of data**.
2. Each **data point is treated as a node**, and the **similarity between points** is represented as an **edge** between nodes.
3. The algorithm uses **eigenvalues and eigenvectors** of this similarity matrix to map data into a **lower-dimensional space** for easier clustering.
4. Once the data is projected, a simple algorithm like **K-Means** is applied on the transformed data to form clusters.
5. Hence, spectral clustering combines the power of **graph theory and linear algebra** for efficient clustering.

3. Connectivity-Based Clustering

1. Traditional clustering depends on **distance**, which fails for non-convex clusters.
2. Spectral clustering, however, depends on **connectivity**, meaning it groups points that are **linked directly or indirectly**.
3. This allows points to belong to the same cluster even if they are not close in terms of geometric distance.
4. Example – In a “two moons” dataset, each moon can be correctly identified as one cluster because points within a moon are connected, even though the distance between moons is small.

Steps in Spectral Clustering

Step 1 – Building the Similarity Graph

1. The first step is to build a **similarity graph** represented as an **adjacency matrix (A)**.
2. This matrix shows how each data point is connected to others.
3. The similarity graph can be built in several ways:

(a) Epsilon-Neighborhood Graph

1. A parameter ϵ (**epsilon**) is selected beforehand.
2. Each point is connected to all points lying within its **epsilon-radius**.
3. If all distances are similar in scale, **edge weights are not stored**, as they add no extra information.
4. The resulting graph is **undirected and unweighted**.
5. Example – In a dataset where $\epsilon = 2$, all points within distance 2 of each other are considered connected.

(b) K-Nearest Neighbors (KNN) Graph

1. A parameter **k** is chosen in advance.
2. An edge is drawn from point **u** to **v** if **v** is among the **k-nearest neighbors** of **u**.
3. This results in a **directed and weighted graph**, because **v** being near **u** doesn't always mean **u** is near **v**.
4. To make it **undirected**, two common approaches are used:
 - (i) Draw an edge if **either** **u** is among **v**'s **k-nearest neighbors** **or** vice versa.
 - (ii) Draw an edge **only if both** **u** and **v** are among each other's **k-nearest neighbors**.
5. Example – If $k = 3$, each point connects to its 3 closest points, forming small local neighborhoods.

(c) Fully Connected Graph

1. In this graph, **each point is connected to every other point**.
2. The edges are **weighted** by the **distance** between two points.
3. This type of graph captures the **global structure** of the data.
4. Example – In image clustering, every pixel is connected to every other pixel, but with smaller weights for distant pixels.

Step 2 – Constructing the Laplacian Matrix

1. Once the similarity matrix is built, the **degree matrix (D)** is computed.
2. The **degree matrix** has diagonal entries representing the total connections of each node.
3. The **Graph Laplacian** is then calculated as:
($L = D - A$), where A is the adjacency matrix.
4. The Laplacian captures the connectivity structure of the entire dataset.

Step 3 – Eigenvalue Decomposition

1. Next, the **eigenvalues and eigenvectors** of the Laplacian matrix are calculated.
2. The **smallest k eigenvectors** are selected to represent the data in a **lower-dimensional space**.
3. These eigenvectors capture the **main connectivity patterns** in the graph.
4. Each row of this transformed matrix represents a data point in the new feature space.

Step 4 – Applying Clustering

1. The transformed data points are then clustered using a simple algorithm like **K-Means**.
2. K-Means now operates in the new space, where clusters are linearly separable.
3. Finally, each cluster in the new space corresponds to a meaningful group in the original data.

10. Example

1. Consider 200 data points forming two **crescent-shaped (moon)** clusters.
2. K-Means fails to separate them because of their curved boundaries.
3. Spectral clustering creates a **connectivity graph** and finds two clusters based on graph structure.
4. Thus, it successfully separates both moon-shaped groups using connectivity rather than distance.

Q. DBSCAN Algorithm (Density-Based Spatial Clustering of Applications with Noise)

=>

1. Introduction

1. DBSCAN is a **density-based clustering algorithm** used to group data points that are close to each other based on a distance measure.
2. It stands for **Density-Based Spatial Clustering of Applications with Noise**.
3. Unlike **K-Means**, DBSCAN does not require you to specify the number of clusters beforehand.
4. It can find clusters of **arbitrary shapes** and can **handle noise (outliers)** efficiently.
5. It is widely used in applications like **geographical data analysis, image segmentation, and anomaly detection**.

2. Key Idea

1. The main idea of DBSCAN is to find areas of **high data density** separated by areas of **low density**.
2. A cluster is formed when there are enough points close to one another based on two parameters:
 - a. **Epsilon (ϵ)** – The radius of the neighborhood around a data point.
 - b. **MinPts** – The minimum number of points required to form a dense region.
3. Points that do not belong to any cluster are considered **noise or outliers**.

3. Types of Points in DBSCAN

1. **Core Points:** A point that has at least **MinPts** points (including itself) within a distance of ϵ .
 - a. Example: If $\epsilon = 2$ and $\text{MinPts} = 4$, a point with 4 or more neighbors within 2 units is a core point.
2. **Border Points:** A point that is **not a core point** but lies within the neighborhood (ϵ) of a core point.
3. **Noise Points:** A point that is **neither a core point nor a border point**, meaning it lies in a low-density region.

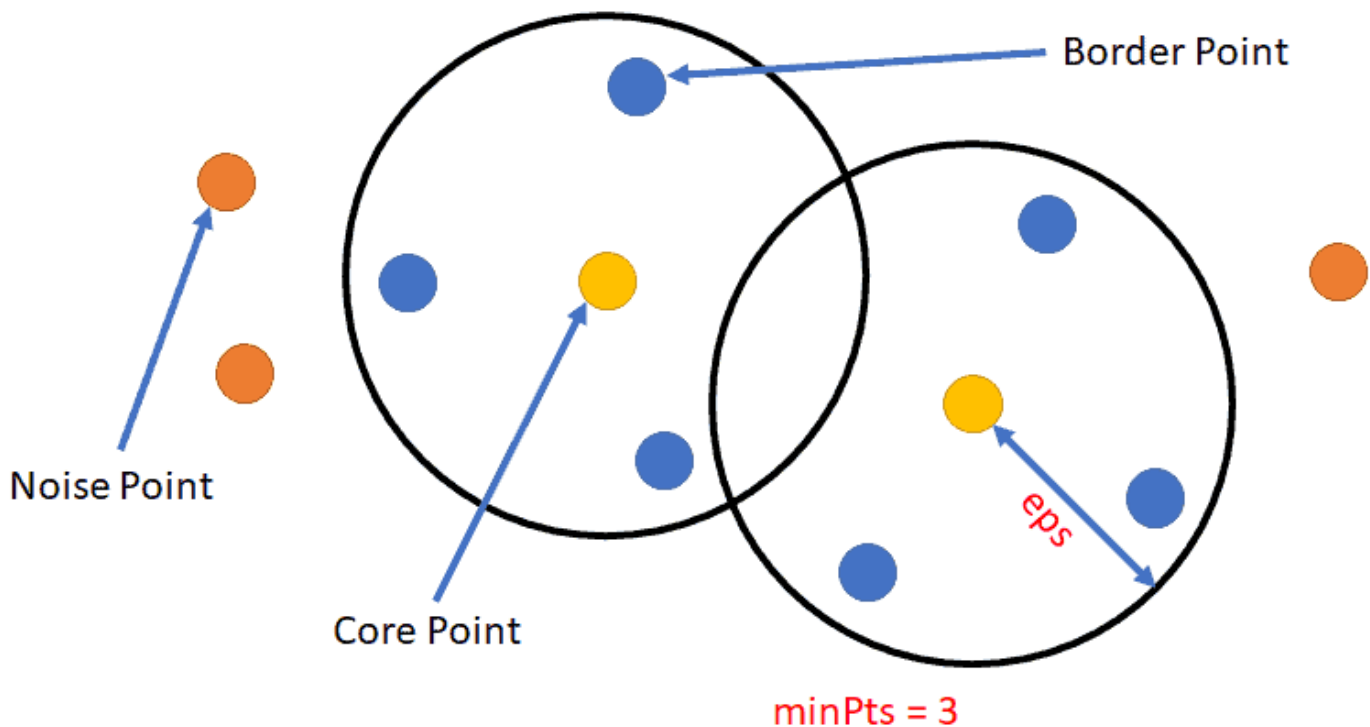
4. Steps in DBSCAN Algorithm

1. **Step 1:** Select an unvisited data point from the dataset.
2. **Step 2:** Find all points within the **ϵ -neighborhood** of that point.
3. **Step 3:**
 - a. If the number of neighbors $\geq \text{MinPts}$, mark the point as a **core point** and form a new cluster.
 - b. Else, mark the point as **noise** temporarily.

4. **Step 4:** Expand the cluster by adding all points that are **density-reachable** (directly or indirectly connected to the core point).
5. **Step 5:** Repeat the process for all remaining unvisited points until all points are labeled as **core**, **border**, or **noise**.

5. Example

1. Suppose you have a dataset of geographical locations of houses.
2. Let $\epsilon = 1$ km and **MinPts = 5**.
3. DBSCAN will identify clusters of houses that are closely packed (within 1 km of each other) and mark isolated houses as **outliers**.
4. This helps in identifying **densely populated regions** and **isolated properties**.



Q. Minimum Spanning Tree (MST)

=>

1. A **Spanning Tree** is a subgraph of a connected, undirected graph that includes all the vertices and has no cycles.
2. It connects all the vertices with the **minimum possible number of edges** — if there are (V) vertices, then the spanning tree will have exactly $(V - 1)$ edges.
3. A **Minimum Spanning Tree (MST)** is a spanning tree that has the **least total edge weight** among all possible spanning trees.
4. MST helps in **reducing complexity** and **minimizing connection cost** while keeping all points connected.
5. MST is widely used in **network design, clustering, image segmentation, and transportation systems**.

2. Properties of a Spanning Tree

1. The number of **vertices (V)** in the spanning tree equals the number of vertices in the graph.
2. The number of **edges (E)** in the spanning tree is always **$V - 1$** .
3. It must be **connected** — there should be only one component.
4. It must be **acyclic** — no closed loops are allowed.
5. The **total cost or weight** of the tree is the sum of the edge weights included.
6. There can be **multiple MSTs** possible for a single graph if several edges have the same weights.

3. Minimum Spanning Tree for Clustering

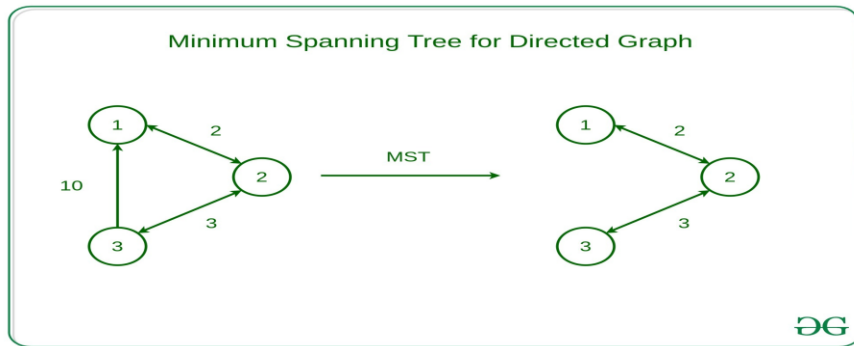
1. MST can be used for **clustering** by treating each data point as a node in a graph.
2. The **distance or dissimilarity** between data points is used as the **edge weight**.
3. MST is first constructed to connect all points using the smallest distances possible.
4. Once the MST is formed, the **longest edges** (which connect distant points or clusters) can be **removed**.
5. The remaining connected components represent **clusters** of similar points.
6. This technique helps to identify **natural groupings** in data without specifying the number of clusters in advance.

Example:

- Suppose you have 6 points representing cities on a map.
- MST connects these cities with the **minimum total road length**.

- If you remove the longest road between two distant groups, you effectively form **two clusters** of nearby cities.

4. Working Example



Step 1: Graph Construction

1. Consider three vertices: 1, 2, and 3.
2. The edges and their weights are:
 - a. Edge (1 → 2) with weight 2
 - b. Edge (2 → 3) with weight 3
 - c. Edge (3 → 1) with weight 10

Step 2: Finding MST

1. To form an MST, we select edges with the **lowest weights** that do **not form a cycle**.
2. The edges selected are:
 - a. (1 → 2) with weight 2
 - b. (2 → 3) with weight 3
3. The total weight = 2 + 3 = **5**, which is the **minimum possible**.
4. The edge (3 → 1) with weight 10 is **ignored** because it forms a cycle and increases cost.

Step 3: Result

1. The MST connects all nodes (1, 2, 3) with the **least total weight (5)**.
2. If used for clustering, the weak connection (weight 10) could be cut, forming **two distinct clusters** based on connectivity.
3. The diagram shows the **original graph** and the **MST** on the right side with minimal edges.

5. Algorithms to Find MST

A. Kruskal's Algorithm

1. It is a **greedy algorithm** that builds the MST by adding edges in increasing order of weight.
2. Steps:
 - a. Sort all edges by their weights.
 - b. Add the smallest edge that does not form a cycle.
 - c. Repeat until there are $(V - 1)$ edges in the tree.
3. It is efficiently implemented using **Disjoint Set Union (DSU)** to track connected components.
4. Example: Used in **network design** or **data clustering** where cost minimization is required.

B. Prim's Algorithm

1. Another **greedy approach**, but it grows the MST one vertex at a time.
2. Steps:
 - a. Start from any vertex.
 - b. At each step, add the smallest edge connecting the current MST to a new vertex.
 - c. Continue until all vertices are included.
3. Best implemented using a **priority queue** to select minimum-weight edges quickly.
4. Example: Used in **routing problems**, **image segmentation**, and **clustering** based on similarity.

C. Boruvka's Algorithm

1. One of the oldest MST algorithms.
2. Starts by treating each vertex as an individual tree.
3. In each iteration, the **cheapest edge** connecting each tree to another is added.
4. Trees merge gradually until one MST remains.
5. Useful for **parallel computing** and **large sparse graphs**.

Q. K-Means Algorithm

=>

1. K-Means is an unsupervised machine learning algorithm used for clustering.
2. Clustering means grouping similar data points together based on patterns.
3. K-Means divides the dataset into **K clusters**, where K is chosen by the user.
4. It is widely used because it is simple, fast, and works well for large datasets.

2. Concept of K-Means

5. The main idea of K-Means is to group data points so that points in the same group are similar.
6. Each group has a center known as a **centroid**.
7. The algorithm tries to minimize the distance between data points and their cluster centroids.
8. The result is a set of compact and well-separated clusters.

3. Steps of the K-Means Algorithm

Step 1: Choose the Number of Clusters (K)

9. Select a value for K, the number of clusters to form.
10. This value may be chosen using methods like the Elbow Method.

Step 2: Initialize Centroids

11. Randomly select K data points as the initial centroids.
12. These centroids act as the starting points for forming clusters.

Step 3: Assign Data Points to Nearest Centroid

13. Calculate the distance from each data point to every centroid.
14. Assign each data point to the cluster with the closest centroid.
15. After this step, all data points belong to one of the K clusters.

Step 4: Recompute Centroids

16. Compute the new centroid of each cluster by taking the mean of all points in that cluster.
17. These new centroids represent updated cluster centers.

Step 5: Repeat Steps 3 and 4

18. Reassign data points to the nearest new centroid.
19. Recalculate centroids again.

20. Repeat until centroids no longer move.

Step 6: Stopping Condition

21. The algorithm stops when clusters become stable.

22. Stability means centroids no longer change or changes are very small.

4. Example of K-Means

Dataset Example

23. Consider a dataset of 10 customers with their purchasing amounts.

24. We want to group them into $K = 2$ clusters: low spenders and high spenders.

Execution

25. Randomly select two initial centroids.

26. Calculate distances of each customer to both centroids.

27. Assign customers to the nearest centroid.

28. Recalculate the mean spending value for each cluster.

29. Update centroids and reassign customers.

30. After a few iterations, the clusters become stable.

Final Output

31. Customers with similar spending patterns form Cluster 1.

32. Customers with high spending form Cluster 2.

5. Advantages of K-Means

33. It is simple and easy to understand.

34. It is computationally efficient and works well for large datasets.

35. It produces compact and easily interpretable clusters.

36. It works well when clusters are spherical and well-separated.

6. Limitations of K-Means

37. The number of clusters (K) must be chosen manually.

38. It may produce poor results if initial centroids are chosen poorly.

39. It does not work well with non-spherical clusters.

40. It is sensitive to outliers and noisy data.

41. Different runs may produce different results due to random initialization.

7. Applications of K-Means

- 42. Customer segmentation in marketing.
- 43. Document clustering in NLP.
- 44. Image compression by grouping similar pixels.
- 45. Grouping locations based on GPS data.
- 46. Detecting patterns in medical datasets.

Q. Expectation–Maximization (EM) Algorithm

=>

1. The Expectation–Maximization (EM) algorithm is an unsupervised learning method.
2. It is used to estimate unknown parameters in probabilistic models.
3. EM is particularly useful when data contains **missing values** or **hidden variables**.
4. It works by repeatedly estimating missing information and improving parameter values.
5. EM is commonly used in clustering, density estimation, and mixture models.

2. Concept of EM Algorithm

6. EM is an iterative optimization technique.
7. The algorithm alternates between two steps: **Expectation (E-step)** and **Maximization (M-step)**.
8. In the E-step, the algorithm estimates the missing or hidden data.
9. In the M-step, the algorithm updates the model parameters based on those estimates.
10. This process continues until the model parameters converge.
11. EM improves the likelihood (fit) of the data at each iteration.

3. Why EM is Needed

12. In many real-world problems, some data is incomplete or partially observed.
13. Traditional algorithms cannot work directly with missing or hidden information.
14. EM overcomes this problem by estimating hidden variables and refining parameters.
15. It is especially powerful for **Gaussian Mixture Models (GMM)** and **soft clustering**.

4. Steps of the EM Algorithm

Step 1: Initialization

16. Start with initial guesses for the model parameters.
17. These parameters may include means, variances, or mixing probabilities.

Step 2: Expectation Step (E-step)

18. Calculate the probability of each data point belonging to each cluster or component.
19. Use current parameter estimates to compute these probabilities.
20. This step “fills in” missing or hidden information.

Step 3: Maximization Step (M-step)

21. Update model parameters using the probabilities computed in the E-step.

- 22. Calculate new means, variances, and mixing weights.
- 23. These new parameters maximize the likelihood of the data.

Step 4: Check for Convergence

- 24. Compare new parameter values with previous ones.
- 25. If the change is very small, the algorithm has converged.
- 26. Otherwise, repeat E-step and M-step.

5. Example: Clustering with Gaussian Mixture Model (GMM)

Dataset

- 27. Consider a dataset of points that appear to come from two overlapping Gaussian distributions.
- 28. We want to cluster them into two groups, but true cluster labels are unknown.

Step-by-step EM Execution

- 29. Step 1: Initialize two cluster centers randomly.
- 30. Step 2 (E-step): For each data point, compute the probability that it belongs to Cluster 1 or Cluster 2.
- 31. Points close to center 1 get higher probability for Cluster 1.
- 32. Points close to center 2 get higher probability for Cluster 2.
- 33. Step 3 (M-step): Update the means of both clusters using these probabilities.
- 34. Update variances and mixing weights too.
- 35. Repeat E-step and M-step until the cluster centers stop moving.

Final Output

- 36. The data gets separated into soft clusters, meaning each point has a probability of belonging to each cluster.
- 37. This helps in handling overlapping data, unlike hard clustering methods such as K-Means.

6. Advantages of EM Algorithm

- 38. EM works well with incomplete or missing data.
- 39. It produces soft clusters, giving more flexible results than K-Means.
- 40. It is easy to implement and mathematically sound.
- 41. EM improves the model likelihood at every iteration.
- 42. It is widely used in statistical modeling and unsupervised learning.

7. Limitations of EM Algorithm

- 43. EM can get stuck in local optima if initialization is poor.
- 44. It can be slow for very large datasets.
- 45. It requires assumptions about the underlying probability distribution.
- 46. Convergence may be slow in high-dimensional problems.

8. Applications of EM Algorithm

- 47. Clustering using Gaussian Mixture Models (GMM).
- 48. Image segmentation and pattern recognition.
- 49. Recommender systems using latent factor models.
- 50. Speech recognition and natural language processing.
- 51. Medical image analysis and anomaly detection.
- 52. Estimation of missing values in datasets.

Q. Distance Metrics Used in Clustering

=>

1. Clustering algorithms group similar data points together based on distance or similarity.
2. Distance metrics measure how far apart two data points are in feature space.
3. The choice of distance metric affects the shape, size, and quality of clusters.
4. Different clustering algorithms use different types of distance measures.
5. Selecting the correct distance metric is essential for achieving meaningful clusters.

2. Euclidean Distance

Meaning

6. Euclidean distance is the straight-line distance between two points in space.
7. It is the most commonly used distance measure in clustering.
8. It works well when the scale of features is uniform.

Properties

9. It calculates distance using the Pythagorean formula.
10. It is sensitive to the magnitude of features.

Example

11. Distance between points (2,3) and (5,7) is the straight-line distance between them.
12. Algorithms like K-Means commonly use Euclidean distance.

3. Manhattan Distance (City Block Distance)

Meaning

13. Manhattan distance calculates the sum of absolute differences between coordinates.
14. It is also known as L1 distance.

Properties

15. It is useful when movement is allowed only horizontally or vertically.
16. It is less sensitive to outliers compared to Euclidean distance.

Example

17. Distance between (2,3) and (5,7) = $|2-5| + |3-7| = 3 + 4 = 7$.
18. Manhattan distance is used in clustering high-dimensional data.

4. Minkowski Distance

Meaning

- 19. Minkowski distance is a generalization of both Euclidean and Manhattan distances.
- 20. It is defined using a parameter “p”.

Properties

- 21. When $p = 1$, Minkowski becomes Manhattan distance.
- 22. When $p = 2$, Minkowski becomes Euclidean distance.
- 23. It is flexible because different values of p adjust the distance behavior.

Example

- 24. For $p = 3$, distances increase more for larger differences in features.

5. Cosine Similarity

Meaning

- 25. Cosine similarity measures the angle between two vectors instead of distance.
- 26. It focuses on direction rather than magnitude.

Properties

- 27. Useful in text clustering and high-dimensional sparse data.
- 28. Two vectors with small angles are considered similar.

Example

- 29. In email or document clustering, cosine similarity checks how similar two documents are based on word frequency.

6. Jaccard Distance

Meaning

- 30. Jaccard distance measures dissimilarity between two sets.
- 31. It is used when data is binary or categorical.

Properties

- 32. Jaccard distance = $1 - (\text{Intersection} / \text{Union})$.

33. It is useful for datasets where features represent presence or absence.

Example

34. For two customers who like different sets of products, Jaccard distance measures how different their preferences are.

7. Hamming Distance

Meaning

35. Hamming distance counts how many positions differ between two strings or binary vectors.

36. It is mainly used for categorical data or string comparison.

Properties

37. Useful in clustering DNA sequences, binary data, and error detection.

38. Measures exact mismatches between values.

Example

39. Distance between "1011" and "1001" is 1 because one position differs.

8. Mahalanobis Distance

Meaning

40. Mahalanobis distance considers the correlations between features.

41. It adjusts distance based on feature variance and covariance.

Properties

42. Useful when features have different scales and correlations.

43. Helps detect outliers in multivariate data.

Example

44. In financial risk analysis, Mahalanobis distance identifies unusual transactions based on correlations.